From 3×3 Pixels to MNIST

A hands-on follow-along workbook for learning image classification from first principles

---

Build it in stages: *pixel grids → vectors → a baseline classifier → neural-network training → real handwritten digits.*

Includes runnable Python labs, checkpoints, exercises, explanations, a study schedule, a glossary, and official references.

Recommended tools: Python · NumPy · Matplotlib · PyTorch

Estimated time: 8–12 focused sessions

How to use this guide

Follow the stages in order. Type the code yourself rather than only reading it. Before looking at a solution, write down what you expect the program to do. Keep a small learning log with three lines after each session:

• **I understood:** …

• **I am still unsure about:** …

• **Next I will try:** …

Learning outcomes

By the end, you should be able to:

• represent a tiny image as numbers;

• explain features, labels, predictions, and classification;

• build and evaluate a simple baseline;

• explain weights, biases, logits, probabilities, loss, gradient descent, and backpropagation at a high level;

• train a neural network on 28×28 MNIST images;

• use a test set correctly and recognize overfitting;

• explain why 3×3 is a teaching tool, not a realistic digit-recognition resolution.

The route

1. Pixels, vectors, and labels
2. Make a 3×3 dataset
3. Build a nearest-prototype baseline
4. Measure performance and inspect mistakes
5. Understand a neuron and a neural network
6. Train a tiny model
7. Move to the real MNIST dataset
8. Train a neural network on MNIST
9. Optional: convolutional neural networks
10. Final project, checklist, glossary, and references

Stage 1 — Pixels are numbers

A grayscale image is a grid of intensity values. For a simple toy example, use `0` for white and `1` for black. Real image datasets often use values from 0 to 255; these are commonly normalized to the range 0–1.

A 3×3 image contains 9 numbers:

```
0 1 0
0 1 0
0 1 0
```

Flattening means reading the grid in a fixed order (usually row by row):

```
[0, 1, 0, 0, 1, 0, 0, 1, 0]
```

That 9-number vector is the model's input. In machine learning, input values are often called **features**. The correct answer we attach to an example is its **label**.

For this first exercise, classify simple patterns rather than actual handwritten digits:

• `vertical`

• `horizontal`

• `diagonal`

This is deliberate: 3×3 images are too low-resolution to preserve enough detail for dependable recognition of all ten digits. Different digits can become identical after shrinking to 3×3. A model cannot recover information that is no longer present.

Lab 1A — Create and display patterns

Create a file named `stage1_pixels.py`:

```
import numpy as np

vertical = np.array([
    [0, 1, 0],
    [0, 1, 0],
    [0, 1, 0],
])

horizontal = np.array([
    [0, 0, 0],
    [1, 1, 1],
    [0, 0, 0],
])

diagonal = np.array([
    [1, 0, 0],
    [0, 1, 0],
    [0, 0, 1],
])

for name, image in [
    ("vertical", vertical),
    ("horizontal", horizontal),
    ("diagonal", diagonal),
]:
    print(name)
    print(image)
    print("flattened:", image.flatten())
    print()
```

Run it:

```
python stage1_pixels.py
```

Checkpoint

■ I can explain why a 3×3 image has 9 values.

■ I can explain why flattening does not change the pixel values.

■ I can explain the difference between a feature and a label.

Exercises

1. Change one pixel in the vertical pattern. What changed in the vector?

2. Write a pattern that is not one of the three classes.

3. Why must every image use the same pixel order when flattened?

Answer hints: A different pixel changes one position in the vector. Consistent ordering is essential because the model learns the meaning of each position.

Stage 2 — Create a tiny dataset

A dataset is a collection of examples. Each example has input features and a known label. The model learns from labeled examples; later, we give it an example whose label is hidden and ask for a prediction.

Lab 2A — Build the dataset

```
import numpy as np

patterns = {
    "vertical": np.array([
        [0, 1, 0],
        [0, 1, 0],
        [0, 1, 0],
    ]),
    "horizontal": np.array([
        [0, 0, 0],
        [1, 1, 1],
        [0, 0, 0],
    ]),
    "diagonal": np.array([
        [1, 0, 0],
        [0, 1, 0],
        [0, 0, 1],
    ]),
}

X = np.stack([image.flatten() for image in patterns.values()])
y = np.array(list(patterns.keys()))

print("X shape:", X.shape)  # (3, 9): 3 examples, 9 features each
print("y shape:", y.shape)  # (3,): 3 labels
print("Labels:", y)
```

`X` conventionally stores the inputs; `y` stores the target labels. Here there are only three examples, so this dataset is a demonstration, not enough to learn robustly.

Lab 2B — Add variation

A model should not only see one perfect version of a pattern. Real images vary: a line may shift, a stroke may be missing, and pixels may be noisy. For a toy project, create a few variations by changing one pixel at a time, then inspect them.

Important: do not generate variations that change the intended class without noticing. A corrupted vertical line can become ambiguous. This is a useful lesson in label quality.

Checkpoint

- ■ I can describe `X` and `y`.
- ■ I understand why a dataset with one example per class is not enough.
- ■ I understand that labels must correctly describe the examples.

Stage 3 — Build a baseline before a neural network

Start with a simple algorithm. A **baseline** gives you something to compare with later. Here, use nearest-prototype classification: compare a new image with each known pattern and choose the closest one.

For binary pixels, Hamming distance counts the positions that differ. For example:

```
A: 0 1 0      B: 0 0 0
   0 1 0         1 1 1
   0 1 0         0 0 0
```

Count the pixel positions where A and B differ. The smaller the distance, the more similar the patterns are.

Lab 3A — Implement the classifier

```
import numpy as np

prototypes = {
    "vertical": np.array([0, 1, 0, 0, 1, 0, 0, 1, 0]),
    "horizontal": np.array([0, 0, 0, 1, 1, 1, 0, 0, 0]),
    "diagonal": np.array([1, 0, 0, 0, 1, 0, 0, 0, 1]),
}

def predict(image_vector):
    distances = {
        label: np.sum(image_vector != prototype)
        for label, prototype in prototypes.items()
    }
    predicted_label = min(distances, key=distances.get)
    return predicted_label, distances

test_image = np.array([
    [0, 1, 0],
    [0, 1, 0],
    [0, 0, 0],
]).flatten()

label, distances = predict(test_image)
print("Distances:", distances)
print("Prediction:", label)
```

What happened?

1. The input is a 9-value vector.

2. The algorithm compares it with each prototype.

3. It counts mismatched pixels.

4. It returns the label with the smallest distance.

This is a hand-designed decision rule, not a neural network. It does not learn its prototypes from data yet.

Exercises

1. Try the exact diagonal pattern. Which class is predicted?

2. Change two pixels in the horizontal pattern. Does the prediction change?

3. Create an ambiguous image. What happens if two classes have the same distance?

4. Modify the code to return the distances as well as the predicted label.

Checkpoint

■ I can explain a baseline.

■ I can explain Hamming distance.

■ I understand that a rule-based classifier and a trained model are not the same thing.

Stage 4 — Evaluate instead of guessing

Accuracy is the fraction of predictions that match the correct labels:

\text{accuracy}=\frac{\text{number of correct predictions}}{\text{number of examples}}

If 8 of 10 examples are classified correctly, accuracy is 0.8 or 80%.

Accuracy alone can be misleading. For instance, a dataset with 95 examples of one class and 5 of another can produce 95% accuracy by always predicting the majority class. Inspect class balance and errors, not just the score.

Lab 4A — Evaluate a list of examples

```
examples = [
    (np.array([0, 1, 0, 0, 1, 0, 0, 1, 0]), "vertical"),
    (np.array([0, 0, 0, 1, 1, 1, 0, 0, 0]), "horizontal"),
    (np.array([1, 0, 0, 0, 1, 0, 0, 0, 1]), "diagonal"),
]

correct = 0
for image, true_label in examples:
    predicted_label, distances = predict(image)
    is_correct = predicted_label == true_label
    correct += int(is_correct)
    print(f"true={true_label:10} predicted={predicted_label:10} correct={is_correct}")

accuracy = correct / len(examples)
print(f"Accuracy: {accuracy:.1%}")
```

This toy evaluation is too small to prove the model generalizes. In a real project, keep separate training and test examples. Do not tune the model repeatedly against the test set; it should be reserved for a final, more honest check.

Checkpoint

■ I can calculate accuracy.

■ I can explain why a tiny test set gives unreliable estimates.

■ I can describe the purpose of a test set.

Stage 5 — Understand the neuron

A simple artificial neuron receives inputs, multiplies each input by a weight, adds a bias, and passes the result through an activation function.

z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b

• **Input (**x**):** pixel values.

• **Weight (**w**):** how strongly an input contributes.

• **Bias (**b**):** a value that shifts the neuron's response.

• **Activation:** a function applied to the weighted sum.

• **Prediction:** the model's output for an input.

At the beginning, weights are usually initialized to small values. The model makes poor predictions. Training changes weights to reduce prediction error.

A neural network connects multiple neurons into layers:

```
9 pixel inputs  →  hidden layer  →  output layer
x1 ... x9       →  learned features → class scores
```

For three toy classes, the output layer can have three scores. For MNIST, it typically has ten scores, one for each digit from 0 to 9.

Lab 5A — Calculate one neuron by hand

Suppose:

• `x = [1, 0, 1]`

• `w = [0.5, -0.2, 0.3]`

• `b = -0.1`

Then:

z = (1)(0.5)+(0)(-0.2)+(1)(0.3)-0.1=0.7

A sigmoid activation converts this score into a value between 0 and 1:

\sigma(z)=\frac{1}{1+e^{-z}}

For `z = 0.7`, the sigmoid is approximately 0.668. This number is not automatically a meaningful class probability in every model; that interpretation depends on the model and training setup.

Mini-quiz

1. What is the role of a weight?

2. What is the role of a bias?

3. Does a network know good weights before training?

4. Why do we use more than one neuron?

Stage 6 — How learning works

Training is a loop:

1. **Forward pass:** make predictions from current weights.

2. **Loss:** quantify how far the predictions are from the correct labels.

3. **Backpropagation:** calculate how each parameter contributed to the loss.

4. **Optimizer step:** update the parameters to reduce loss.

5. Repeat over many batches and epochs.

An **epoch** means one pass through the training dataset. A **batch** is the subset of examples processed together. The **learning rate** controls the size of parameter updates.

```
training examples
      ↓
forward pass → predictions
      ↓
compare with true labels → loss
      ↓
backpropagation → gradients
      ↓
optimizer updates weights
      ■ repeat
```

A low training loss does not guarantee good performance on unseen images. If the model memorizes training examples but fails on new ones, it is **overfitting**.

Optional NumPy experiment: logistic regression

This is a small learning experiment, not a replacement for the later MNIST neural network. It trains a linear classifier on the three ideal patterns. Because the dataset is tiny and the patterns are intentionally simple, treat it as a way to inspect training mechanics, not as a meaningful benchmark.

```
import numpy as np

X = np.array([
    [0,1,0, 0,1,0, 0,1,0],  # vertical
    [0,0,0, 1,1,1, 0,0,0],  # horizontal
    [1,0,0, 0,1,0, 0,0,1],  # diagonal
], dtype=float)

# One-hot labels: vertical, horizontal, diagonal
y = np.eye(3)

rng = np.random.default_rng(7)
W = rng.normal(0, 0.1, size=(9, 3))
b = np.zeros((1, 3))
learning_rate = 0.1

def softmax(logits):
    shifted = logits - logits.max(axis=1, keepdims=True)
    exp_scores = np.exp(shifted)
    return exp_scores / exp_scores.sum(axis=1, keepdims=True)
```

```
for step in range(1000):
    logits = X @ W + b
    probs = softmax(logits)

    # Average cross-entropy loss
    loss = -np.mean(np.sum(y * np.log(probs + 1e-12), axis=1))

    # Gradients for softmax + cross-entropy
    d_logits = (probs - y) / len(X)
    dW = X.T @ d_logits
    db = d_logits.sum(axis=0, keepdims=True)

    W -= learning_rate * dW
    b -= learning_rate * db

    if step % 200 == 0:
        print(f"step={step:4} loss={loss:.4f}")

predicted = np.argmax(softmax(X @ W + b), axis=1)
print("predicted class indices:", predicted)
print("true class indices:     ", np.argmax(y, axis=1))
```

What to inspect: Does loss trend downward? Do predicted class indices match the labels? Try a different learning rate. If it is too large, training may jump around or diverge; if too small, progress can be slow.

Caution: This code is a teaching example with only three training samples. It does not establish that the model will work on new examples.

Stage 7 — Prepare your Python environment

Create a project folder:

```
pixel-to-mnist/
■■■ stage1_pixels.py
■■■ stage2_dataset.py
■■■ stage3_baseline.py
■■■ stage4_evaluation.py
■■■ stage6_logistic_regression.py
■■■ mnist_dense.py
■■■ mnist_cnn.py
■■■ README.md
```

Create and activate a virtual environment.

Windows PowerShell:

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install numpy matplotlib scikit-learn torch torchvision
```

macOS/Linux:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install numpy matplotlib scikit-learn torch torchvision
```

PyTorch installation commands can differ by operating system and hardware. If the command above does not work, use the official selector at https://pytorch.org/get-started/locally/.

Verify your setup:

```
python -c "import numpy, torch; print('NumPy', numpy.__version__); print('PyTorch',
    torch.__version__)"
```

You do not need a GPU for the first MNIST experiments.

Stage 8 — Load and inspect MNIST

MNIST consists of 28×28 grayscale images of handwritten digits, with labels from 0 to 9. A flattened image has 784 input values.

For a beginner, `torchvision.datasets.MNIST` is a practical way to download and load the dataset.

Lab 8A — Load and display samples

```
import matplotlib.pyplot as plt
from torchvision import datasets, transforms

transform = transforms.ToTensor()

train_data = datasets.MNIST(
    root="data",
    train=True,
    download=True,
    transform=transform,
)

test_data = datasets.MNIST(
    root="data",
    train=False,
    download=True,
    transform=transform,
)

print("Training examples:", len(train_data))
print("Test examples:", len(test_data))

image, label = train_data[0]
print("Image tensor shape:", image.shape)  # [1, 28, 28]
print("Label:", label)

plt.imshow(image.squeeze(0), cmap="gray")
plt.title(f"Label: {label}")
plt.axis("off")
plt.show()
```

`ToTensor()` converts image pixels to a tensor and typically scales 8-bit pixel intensities to the range 0–1.

Inspect several examples

```
fig, axes = plt.subplots(2, 5, figsize=(10, 4))
for i, ax in enumerate(axes.flat):
    image, label = train_data[i]
    ax.imshow(image.squeeze(0), cmap="gray")
    ax.set_title(f"Label: {label}")
    ax.axis("off")
plt.tight_layout()
plt.show()
```

Checkpoint

■ I can explain why MNIST inputs have 784 pixels.

■ I can display an image and its label.

■ I know that the label is the target, not another input feature.

■ I understand the difference between training and test data.

Stage 9 — Train a dense neural network on MNIST

A dense neural network is a useful first deep-learning model. It flattens each 28×28 image into 784 values, processes those values through a hidden layer, and outputs ten class scores.

Lab 9A — Complete training script

Save as `mnist_dense.py`:

```
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

torch.manual_seed(7)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
print("Device:", device)

transform = transforms.ToTensor()
train_data = datasets.MNIST(
    "data", train=True, download=True, transform=transform
)
test_data = datasets.MNIST(
    "data", train=False, download=True, transform=transform
)

train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
test_loader = DataLoader(test_data, batch_size=256, shuffle=False)

class DigitMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.network = nn.Sequential(
            nn.Flatten(),          # [batch, 1, 28, 28] -> [batch, 784]
            nn.Linear(784, 128),
            nn.ReLU(),
            nn.Linear(128, 10),    # 10 class scores (logits)
        )

    def forward(self, x):
        return self.network(x)

model = DigitMLP().to(device)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

def evaluate(model, loader):
    model.eval()
    total = 0
    correct = 0
    total_loss = 0.0

    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            logits = model(images)
            loss = loss_fn(logits, labels)
            total_loss += loss.item() * labels.size(0)
            predictions = logits.argmax(dim=1)
            correct += (predictions == labels).sum().item()
            total += labels.size(0)

    return total_loss / total, correct / total

for epoch in range(5):
    model.train()
    running_loss = 0.0
    seen = 0

    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        logits = model(images)
        loss = loss_fn(logits, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * labels.size(0)
        seen += labels.size(0)

    train_loss = running_loss / seen
    test_loss, test_accuracy = evaluate(model, test_loader)
    print(
        f"Epoch {epoch + 1}: "
        f"train_loss={train_loss:.4f}, "
        f"test_loss={test_loss:.4f}, "
        f"test_accuracy={test_accuracy:.2%}"
    )
```

Run:

```
python mnist_dense.py
```

The first run downloads the dataset. Runtime depends on your hardware. Your exact accuracy will vary. The goal is to understand the pipeline, not chase a particular number.

Understand the key lines

• `nn.Flatten()` converts the image to a vector of 784 values.

• `nn.Linear(784, 128)` learns weights and biases for a hidden layer.

• `nn.ReLU()` adds a non-linear activation.

• `nn.Linear(128, 10)` produces ten logits.

• `CrossEntropyLoss()` compares logits with integer class labels.

• `loss.backward()` computes gradients through backpropagation.

• `optimizer.step()` updates the parameters.

• `model.eval()` switches off training-specific behavior.

• `torch.no_grad()` avoids storing gradients during evaluation.

Do not add a softmax layer before `CrossEntropyLoss`; PyTorch's loss function expects logits and handles the appropriate numerical operation internally.

Experiments

Change one variable at a time and record what happens:

1. Train for 1 epoch, then 5 epochs.

2. Change the hidden layer from 128 units to 32 or 256.

3. Try a batch size of 32 or 128.

4. Change the learning rate to 0.01, then 0.0001.

5. Save the results in a table with the change, training loss, test accuracy, and your explanation.

Avoid repeatedly tuning against the test set. For serious model selection, make a validation split from the training data and keep the official test set for the final evaluation.

Stage 10 — Inspect errors

Accuracy tells you how often the model is correct, but individual mistakes tell you what it struggles with.

Add this after training in `mnist_dense.py`:

```
import matplotlib.pyplot as plt

model.eval()
wrong_images = []
wrong_labels = []
wrong_predictions = []

with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        logits = model(images)
        predictions = logits.argmax(dim=1)
        wrong = predictions != labels

        for image, label, prediction in zip(
            images[wrong], labels[wrong], predictions[wrong]
        ):
            wrong_images.append(image.cpu().squeeze(0))
            wrong_labels.append(label.item())
            wrong_predictions.append(prediction.item())

        if len(wrong_images) >= 10:
            break

fig, axes = plt.subplots(2, 5, figsize=(10, 5))
for i, ax in enumerate(axes.flat):
    ax.imshow(wrong_images[i], cmap="gray")
    ax.set_title(f"True {wrong_labels[i]} / Pred {wrong_predictions[i]}")
```

```
    ax.axis("off")
plt.tight_layout()
plt.show()
```

This loop may collect more than ten mistakes in the final batch, but displays the first ten. If the model makes fewer than ten mistakes in the batches examined, adapt the code to handle that case.

Ask:

• Which digit pairs are confused?

• Is the writing ambiguous?

• Is the image unusually faint or thick?

• Does the model appear to rely on overall shape?

• What additional evidence would help you decide whether a change improves the model?

Stage 11 — Optional: build a convolutional neural network

Dense networks treat the flattened image as a list of features. A **convolutional neural network (CNN)** also uses the spatial arrangement of pixels. Small learned filters detect local patterns; later layers combine them into more complex patterns.

A simple architecture might be:

```
28×28 image
    ↓
convolution + ReLU
    ↓
pooling
    ↓
convolution + ReLU
    ↓
pooling
    ↓
flatten
    ↓
fully connected layer
    ↓
10 output logits
```

Once the dense model works, try this CNN:

```
class DigitCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),  # 28x28 -> 14x14
            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),  # 14x14 -> 7x7
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 7 * 7, 64),
            nn.ReLU(),
            nn.Linear(64, 10),
        )

    def forward(self, x):
        return self.classifier(self.features(x))
```

To train this model, replace `model = DigitMLP()` with `model = DigitCNN()`. Keep the same loss, optimizer, training loop, and evaluation function.

Compare the models

Record:

• test accuracy;

• training time;

• number of parameters (optional);

• types of mistakes;

• how much harder the architecture is to understand.

A more complex model is not automatically better for every problem. Model choice depends on the task, data, compute budget, and maintenance cost.

Final project — Your own mini digit recognizer

Complete these tasks in order:

■ Create at least three 3×3 pattern classes.

■ Convert images to flattened vectors.

■ Build a nearest-prototype baseline.

■ Write tests for known examples.

■ Train the NumPy logistic-regression model.

■ Explain the difference between a hand-coded rule and learned parameters.

■ Load MNIST and display samples.

■ Train the dense neural network.

■ Record train loss and test accuracy for each epoch.

■ Inspect at least ten incorrect predictions if available.

■ Train the CNN and compare results.

■ Write a one-page summary of what you learned.

Suggested study schedule

Session	Focus	Deliverable
1	Pixels and vectors	`stage1_pixels.py`
2	Dataset and labels	`stage2_dataset.py`
3	Baseline classifier	`stage3_baseline.py`
4	Accuracy and errors	Evaluation notes
5	Neurons and weights	Hand-calculated neuron
6	Loss and gradient descent	NumPy logistic regression
7	Environment and MNIST	Displayed sample images
8	Dense network	`mnist_dense.py`
9	Error analysis	Misclassification gallery
10	CNN and comparison	`mnist_cnn.py` and report

You can spend two or three sessions on any stage that feels difficult. Understanding the ideas is more important than finishing quickly.

Glossary

• **Pixel:** one cell in an image grid.

• **Feature:** a numeric input used by a model.

• **Label/target:** the correct class attached to an example.

• **Dataset:** a collection of examples.

• **Classifier:** a model or rule that assigns class labels.

• **Training:** adjusting model parameters using examples.

• **Inference:** using a trained model to make predictions.

• **Weight:** a learned parameter controlling an input's contribution.

• **Bias:** a learned offset.

• **Logit:** a raw class score before probability conversion.

• **Loss:** a number measuring prediction error for training.

• **Gradient:** the direction and rate of change of loss with respect to parameters.

• **Gradient descent:** updating parameters to reduce loss.

• **Backpropagation:** efficiently calculating gradients through a network.

• **Epoch:** one pass through the training dataset.

• **Batch:** a group of examples processed together.

• **Overfitting:** fitting training data too closely and failing to generalize.

• **Generalization:** performing well on examples not used for training.

• **CNN:** a neural network architecture designed to exploit spatial structure in images.

Common beginner mistakes

1. **Starting with a deep-learning library too early.** First understand inputs, labels, and a simple baseline.
2. **Confusing training accuracy with real-world accuracy.** Use held-out data.
3. **Changing many settings at once.** Change one thing and record the effect.
4. **Assuming 3×3 is enough for digit recognition.** It is a toy representation; many digit details disappear.
5. **Expecting loss to decrease perfectly every step.** Mini-batch training can fluctuate.
6. **Using softmax before PyTorch** `CrossEntropyLoss`**.** Feed it raw logits.
7. **Tuning on the test set.** Use a validation split for model selection.
8. **Only looking at the final accuracy.** Inspect examples the model gets wrong.

References

• **MNIST database:** Yann LeCun's page — http://yann.lecun.com/exdb/mnist/

• **PyTorch tutorials:** https://pytorch.org/tutorials/

• **Torchvision MNIST dataset API:** https://pytorch.org/vision/stable/generated/torchvision.datasets.MNIST.html

• **PyTorch** `CrossEntropyLoss`**:** https://pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html

• **NumPy quickstart:** https://numpy.org/doc/stable/user/quickstart.html

• **Matplotlib image display:** https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.imshow.html

Completion reflection

Write a few sentences in your own words:

1. What does the model receive as input?

2. What does training change?

3. What is the difference between a logit and a label?

4. Why do we evaluate on unseen data?

5. Why is a CNN useful for images?

6. What was the most confusing concept, and how would you explain it now?

You have completed the guide when you can explain the entire path from pixel grid → numeric input → prediction → loss → parameter update → evaluation without simply repeating the code.